# Deterministic Sobol European Option Pricing

A white paper on the fused Sobol, Gaussian transform, exponential payoff, and reduction path

**Executive summary**

The engine uses the deterministic order of a dimension-1 Sobol sequence to avoid general-purpose random-number post-processing. For most lanes, it does not store Gaussian values and does not call a runtime exponential. Instead, it evaluates a small local payoff polynomial whose coefficients are prepared once for the contract.

Private technical note. Generated from the current AVX-512 pricing prototype.

## Contents

# 1 The Pricing Problem

A European option has a payoff at one future time. In the Black–Scholes model, the terminal stock price is

$$S_T = S_0 \exp\left((r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}\,Z\right),$$

where $Z$ is a standard normal random variable. The discounted payoff is

$$C = e^{-rT}\max(S_T - K, 0) \quad \text{for a call,}$$

and

$$P = e^{-rT}\max(K - S_T, 0) \quad \text{for a put.}$$

In a Monte Carlo or quasi-Monte Carlo engine, the normal variable $Z$ usually comes from a uniform number $u \in [0, 1]$ through the inverse normal CDF:

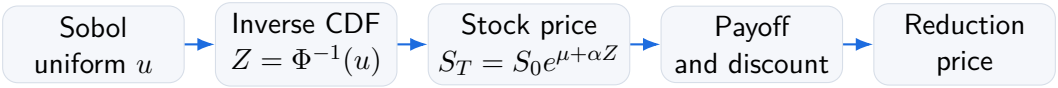
$$Z = \Phi^{-1}(u).$$

**What the engine changes**

The usual calculation creates or streams intermediate values: Sobol uniforms, Gaussian values, exponentials, payoffs, and finally a sum. The new engine collapses most of those stages. In the steady-state path, a Sobol lane is transformed directly into a discounted payoff contribution.

# 2 The Conventional Path

The conventional path is easy to understand but expensive:




This path is robust and general. It is also doing work that is not always needed for a fixed, deterministic Sobol layout:

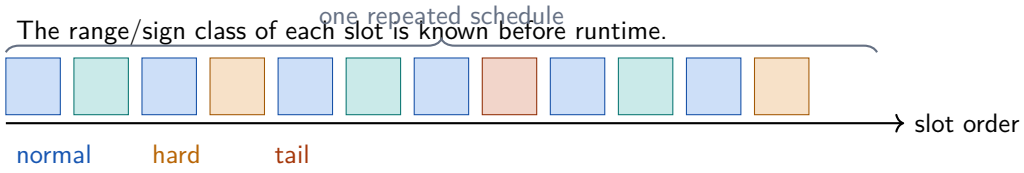
- The inverse CDF is a nonlinear function with expensive tail handling.
- The exponential is another nonlinear function.
- Intermediate arrays create memory traffic.
- General-purpose libraries must handle arbitrary input order and arbitrary distributions.

# 3 The Key Observation: Deterministic Sobol Slots

Sobol points are not random in the usual sense. They follow a deterministic digital-net pattern. The current engine uses dimension 1 and a public block size of 8192 samples. Internally, each public block is processed as two 4096-sample chunks, and each hot loop step handles two AVX-512 registers:

$$2 \times 16 = 32 \text{ float values per store-pair slot.}$$

For all steady-state chunks after the prefix, the relevant ranges repeat in a known order. That makes the following possible:




The important point is not that the sequence is simple. It is that the engine knows where it is in the sequence. That lets the implementation avoid asking, for every lane, "which range am I in?".

## 4 Mantissa Injection

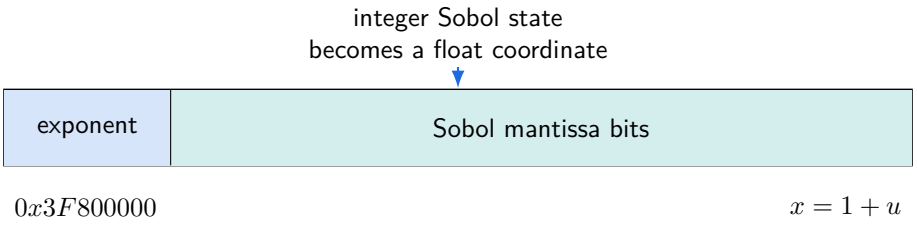
A 32-bit floating-point value in $[1, 2)$ has exponent bits fixed to 127, and the mantissa carries the fractional position. The engine takes the top bits of the Sobol integer and injects them into a float with exponent 1:

$$x = 1 + u, \quad x \in [1, 2).$$

In assembly this is the familiar pattern:

$$\text{shift Sobol integer} \rightarrow \text{OR with } 0x3F800000.$$

That is cheaper than fully normalizing to $u \in [0, 1]$, and it gives the local polynomial a stable input coordinate.




## 5 From Gaussian Transform to Direct Payoff

The earlier Gaussian path used scheduled linear coefficients for normal slots:

$$Z(x) \approx g_0 + g_1 x.$$

The European payoff needs

$$\exp(\mu + \alpha Z(x)), \quad \alpha = \sigma\sqrt{T}, \quad \mu = (r - \tfrac{1}{2}\sigma^2)T.$$

For a call option, after discounting and factoring out constants:

$$\text{payoff}(x) = \max\left(A \exp(\alpha(g_0 + g_1 x)) - B,\ 0\right),$$

where

$$A = e^{-rT} S_0 e^{\mu}, \qquad B = e^{-rT} K.$$

For a put, the same shape is used with signed coefficients:

$$\text{payoff}(x) = \max\left(-A\exp(\alpha(g_0 + g_1 x)) + B,\ 0\right).$$

Instead of evaluating the exponential in the hot loop, contract setup composes the already chosen exponential polynomial with the Gaussian affine model. Let

$$E(t) \approx e^t$$

be the baked polynomial used by the fused kernel, and let $m$ be the deterministic center of this Sobol slot. Define

$$t_m = \alpha(g_0 + g_1 m), \qquad s = \alpha g_1.$$

For a call away from the strike kink, the discounted local payoff is

$$f(x) = A\,E(t_m + s(x - m)) - B.$$

The quadratic coefficients follow directly from derivatives of $E$:

$$p_0 = A\,E(t_m) - B,$$

$$p_1 = A\,E'(t_m)\,s,$$

$$p_2 = \tfrac{1}{2}A\,E''(t_m)\,s^2.$$

For a put, the same formulas are used with the signed scale and offset:

$$f(x) = -A\,E(t_m + s(x - m)) + B.$$

If the small local cell crosses the payoff boundary, setup handles that slot as a kink cell using the same baked polynomial; it still avoids per-slot library exponential calls.

In both cases, the hot loop only needs

$$f(x) \approx p_0 + p_1(x - m) + p_2(x - m)^2.$$

**The core hot-loop idea**

For normal slots, the expensive composition

$$u \to Z \to \exp(\mu + \alpha Z) \to \text{payoff}$$

is replaced by a local polynomial directly in the mantissa-injected Sobol coordinate $x = 1 + u$.

## 6 Numerical Example

Use the standard benchmark contract:

$$S_0 = 100, \quad K = 100, \quad r = 0.05, \quad \sigma = 0.2, \quad T = 1.$$

Then

$$\mu = (0.05 - 0.5 \cdot 0.2^2) \cdot 1 = 0.03,$$

$$\alpha = 0.2\sqrt{1} = 0.2,$$

$$e^{-rT} = e^{-0.05} \approx 0.951229.$$

For a call, the setup scale is

$$A = e^{-rT} S_0 e^{\mu} \approx 0.951229 \cdot 100 \cdot e^{0.03} \approx 98.0199,$$

and

$$B = e^{-rT} K \approx 95.1229.$$

Suppose a deterministic slot has local center

$$m = 1.625.$$

If its Gaussian affine model for this slot is

$$Z(x) \approx -0.80 + 0.50x,$$

then at the center:

$$Z(m) \approx -0.80 + 0.50 \cdot 1.625 = 0.0125.$$

The scaled exponent at the center is

$$t_m = \alpha Z(m) = 0.2 \cdot 0.0125 = 0.0025.$$

The call payoff model at the center is

$$\max(98.0199 \cdot E(0.0025) - 95.1229, 0) \approx 3.142.$$

The first derivative coefficient is approximately

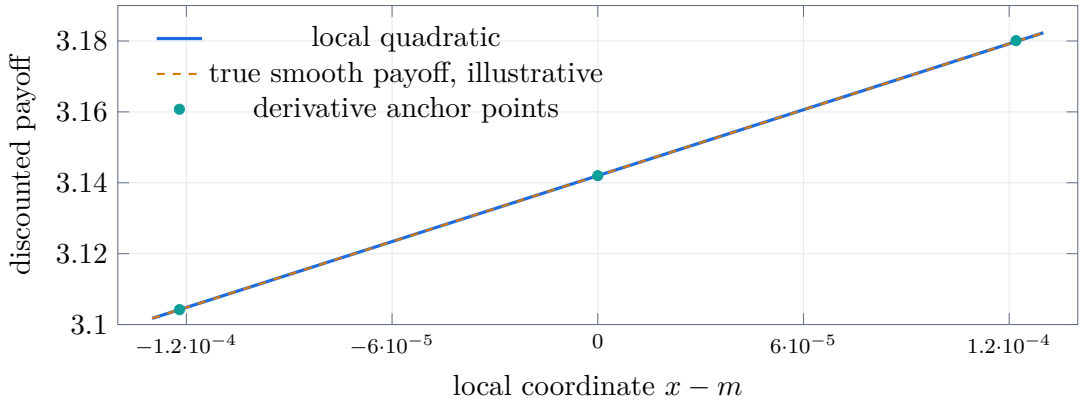
$$p_1 \approx 98.0199 \cdot E'(0.0025) \cdot (0.2 \cdot 0.50),$$

and the curvature coefficient is approximately

$$p_2 \approx \tfrac{1}{2} \cdot 98.0199 \cdot E''(0.0025) \cdot (0.2 \cdot 0.50)^2.$$

The real engine computes these coefficients for every deterministic slot at contract setup by algebraically composing the polynomial pieces. The assembly then processes each lane with a subtract, two fused multiply-add instructions, a max against zero, and an accumulation.

## 7 Visualizing the Local Approximation




The figure is illustrative, but it captures the engineering reason the method works: each local interval is tiny. The quadratic is not trying to approximate the whole payoff function. It only approximates the payoff over the small neighborhood that a deterministic Sobol slot can visit.

## 8 Where the First Block and Tails Differ

The first Sobol prefix is special. It contains the origin and early points whose range schedule does not match the steady-state repeated schedule. The engine therefore patches known first-block vectors.

The extreme tail slots are also special. The inverse normal CDF changes quickly near 0 and 1. For those lanes, the current implementation keeps a safer Gaussian/tail path instead of forcing everything through the direct quadratic normal-slot path.

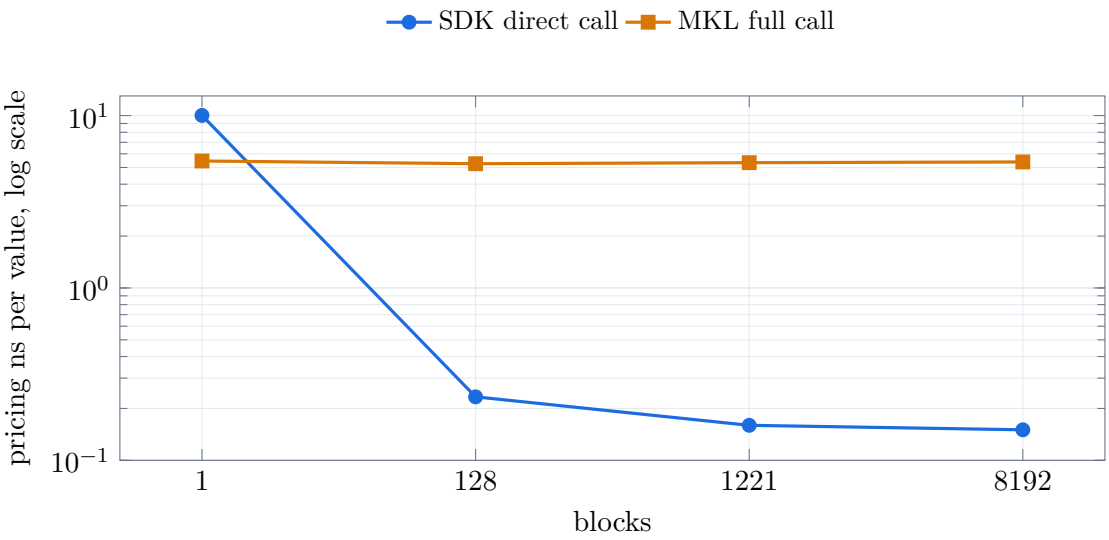
**Why one-block runs look weak**

At one block, the special prefix and setup overhead are a large part of the entire timed run. At many blocks, the steady-state path dominates and the fixed cost is amortized.

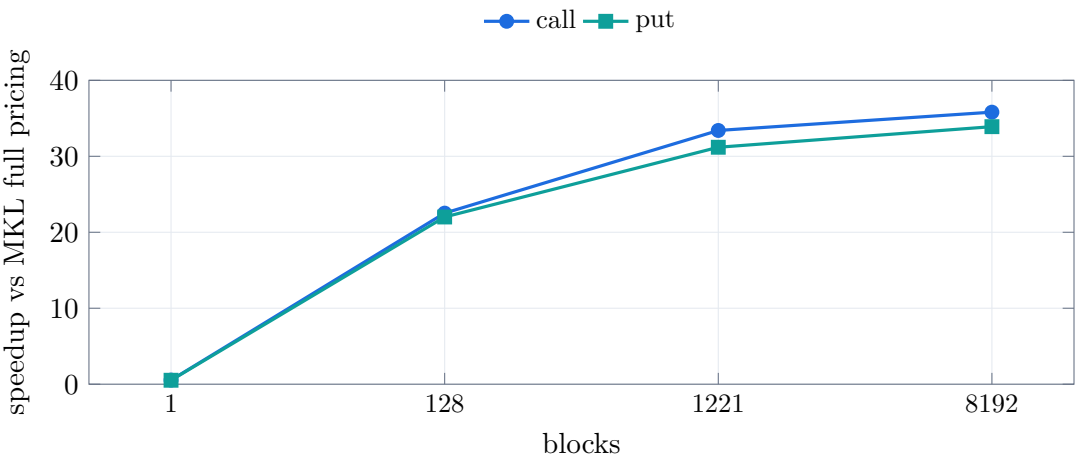
## 9 Benchmark Snapshot

The following results were run on an AWS EC2 instance. The exact instance type and CPU model were not captured in the snapshot.

| Type | Blocks | Samples | SDK ns/value | MKL full | MKL Sobol | vs full | vs Sobol |
|---|---|---|---|---|---|---|---|
| call | 1 | 8,192 | 10.023682 | 5.451050 | 0.109009 | 0.54x | 0.01x |
| put | 1 | 8,192 | 9.969727 | 5.200562 | 0.109131 | 0.52x | 0.01x |
| call | 128 | 1,048,576 | 0.233498 | 5.257521 | 0.188629 | 22.52x | 0.81x |
| put | 128 | 1,048,576 | 0.227925 | 5.016124 | 0.187053 | 22.01x | 0.82x |
| call | 1221 | 10,002,432 | 0.159627 | 5.331564 | 0.201620 | 33.40x | 1.26x |
| put | 1221 | 10,002,432 | 0.163024 | 5.083920 | 0.192128 | 31.19x | 1.18x |
| call | 8192 | 67,108,864 | 0.150344 | 5.383434 | 0.344888 | 35.81x | 2.29x |
| put | 8192 | 67,108,864 | 0.148991 | 5.051132 | 0.349008 | 33.90x | 2.34x |




The log-scale chart shows the pricing transition clearly. The SDK path is weak for a single block, becomes competitive quickly, and then reaches its steady-state advantage as more deterministic blocks are processed. The MKL Sobol uniform column in the table is generator-only context; it does not include the Gaussian transform or payoff.




## 10 What the Assembly Actually Does in the Fast Path

For a normal two-register store-pair, the direct path is conceptually:

1. update the Sobol integer state for two $zmm$ registers,
2. keep the top Sobol bits,
3. inject them into the mantissa to get $x = 1 + u$,
4. subtract the slot center $m$,
5. evaluate the local payoff polynomial,
6. clamp the payoff at zero,
7. add the result to the vector accumulator.

In symbolic assembly form, the normal pricing part is:

$$\begin{aligned} dx &\leftarrow x - m, \\ y &\leftarrow p_2, \\ y &\leftarrow y \cdot dx + p_1, \\ y &\leftarrow y \cdot dx + p_0, \\ y &\leftarrow \max(y, 0), \\ sum &\leftarrow sum + y. \end{aligned}$$

That is the essence of the speedup. The hot loop is no longer a general inverse normal plus exponential. It is a deterministic local pricing evaluator.

## 11 Limitations and Next Steps

- The current direct payoff path is specialized to dimension 1.
- The first Sobol prefix still needs patching.
- Tail slots still use the safer Gaussian/exp path.

- Very small runs are dominated by setup and entry overhead.
- The benchmark snapshot should be repeated with the exact CPU model and pinned frequency settings recorded.

The method is strongest when pricing many block-aligned paths for the same contract. That is exactly the target shape for high-throughput option pricing: prepare the contract once, then run many deterministic Sobol blocks with minimal memory traffic and minimal nonlinear math in the hot loop.